

Tatami — Phase 1

Shredding JSON into OCaml Modules: the Lean implementation

Durwasa Chakraborty

Vimala Soundarapandian

Vishakh Desai

Working notes

1 What this document is

This describes the implementation of Phase 1: the program that reads JSON and writes OCaml module signatures. The program is written in Lean 4 and compiled to a binary.

The authority on *what* the generated modules look like is the design note, `tatami.pdf`. Its §2.1 fixes the correspondence between a relational schema and an OCaml module, and where it speaks we follow it exactly. It is, however, silent on most of what an implementation must decide: it says nothing about how a member seen as both an integer and a float should be typed, nothing about member names that are not legal OCaml identifiers, and nothing about which inputs to refuse. Everything of that kind is ours.

The two are marked throughout: `[design note]` for what the note fixes, `[our choice]` for what we decided.

An earlier version of this document described a different approach, in which the shredder was an OCaml program specified by twenty-two numbered scenarios. That approach was abandoned; the numbering it introduced (S0–S22, C1–C6) appears nowhere in the current sources and should not be used. This document replaces it entirely.

2 Why the implementation is in Lean

The generator is a Lean program compiled to a native binary. It reads JSON on standard input or from a file and writes OCaml text.

This is not extraction. Lean has no OCaml backend, and none is needed. We are not extracting Lean code into OCaml; we are *generating* OCaml text, which is ordinary data our program produces. The obstacle that would apply to extraction does not arise.

The output is a datatype, not a string. The generator’s result type is `Ocaml.File` — a small inductive type expressing exactly the declarations the design note’s signature contains, and nothing else. It cannot express a function body or any expression at all.

That indirection is the point. Properties of the generated code are statable about a datatype and are not statable about a string: “every record field name is distinct” is a proposition about `Ocaml.File`, whereas about a `String` it would first require writing an OCaml parser to say anything at all. Turning the datatype into text is a separate, deliberately dull step.

What stays trusted. Three things, regardless of what is proved:

- the printer, `File → String`, which is where proofs stop and text begins;

- any model we write of OCaml’s own rules — we can check the generated code against *our* account of what OCaml accepts, not against the compiler;
- the reader, `String → Doc`, which is the same kind of boundary as the printer at the other end.

3 What the design note fixes

§2.1, Step 4 gives the correspondence, and we follow all of it:

Relational	OCaml
table	module
row	value of type <code>t</code>
column	record field
primary key	abstract type <code>id</code>
foreign key	field holding another module’s <code>id</code>
joining on it	accessor function
NOT NULL	plain field
nullable	option field

Step 3 shows the generated signature for one worked example. Our output for that input is byte-for-byte identical to the listing in the note:

```
{ "a": 1, "b": { "c": 1, "d": 2 } }
```

```
(* generated.mli *)

module B : sig
  type id
  type t = { id : id; c : int; d : int }
  val get : id -> t
end

module Root : sig
  type id
  type t = { id : id; a : int; b : B.id }
  val get : id -> t
  val b : t -> B.t
end
```

The note also states, without showing it, that “nested and repeated substructures are normalized into separate relations linked by generated keys, with foreign-key constraints materialized rather than left implicit”. The nested case is the example above. The repeated case is §6.

4 Scalars

Numbers. [our choice] JSON has one number type; OCaml has `int` and `float`. A member is `int` only if every value seen there was written without a decimal point *and* fits OCaml’s native `int`; otherwise it is `float`.

```
{ "qty": 1 } { "qty": 2 } -> qty : int
{ "p": 1 } { "p": 2.5 } -> p : float (widened)
{ "x": 1.0 } -> x : float (written with a point)
{ "id_no": 12345678901234567890 } -> id_no : float (outside native range)
```

The distinction is lexical rather than by value, so 1.0 is a float even though its value is round. That is recoverable from Lean’s parser, which records a number as a mantissa and an exponent: a decimal point shows up as a non-zero exponent.

Strings and booleans. [our choice] `string` and `bool`, with nothing to decide.

Reading JSON. [our choice] We read JSON ourselves rather than using `Lean.Json`, for two reasons. Its parser stores an object as a tree map, so a repeated member is discarded before anything can refuse it, and the order of the rest is lost. It also normalises numbers, so the text a number was written with cannot be recovered — and the distinction between 1 and 1.0 rests on exactly that.

Our reader keeps members in order with repeats intact, and keeps each number as the literal it was written with.

Null and absence. [our choice] Both make a member partial, and the generated type carries `option`, which is the note’s rule for a nullable column. The two are not distinguished in the type: a corpus where a member is sometimes `null` and one where it is sometimes missing produce the same module. The counts are kept separately in the report (§9) precisely because the type cannot carry them.

A member never given a type. [our choice] A member present in every document but always `null` has no observed type. It is kept, as `unit option` — the type then records both facts that are true: the member exists, and nothing is known about it.

5 Objects

A nested object becomes a table of its own, named for its path, and the parent holds a key into it with an accessor that follows the key. [design note] This is exactly the note’s example, generalised to any depth. [our choice]

```
{ "a": 1, "b": { "c": 1, "inner": { "z": true } } }
```

```
module B_inner : sig
  type id
  type t = { id : id; z : bool }
  val get : id -> t
end

module B : sig
  type id
  type t = { id : id; c : int; inner : B_inner.id }
  val get : id -> t
  val inner : t -> B_inner.t
end

module Root : sig ... val b : t -> B.t end
```

Module names come from paths. [our choice] A table is identified by its path from the root, and its module is named for the member names along that path, mangled and joined: the root is `Root`, `.b` is `B`, `.b.inner` is `B_inner`.

Distinct tables therefore cannot collide, and the reason is worth recording because it rests on the mangling: paths are distinct by construction, mangling is injective, and it escapes _

as `_5f`, so a literal underscore in a member name can never be mistaken for the separator joining two path segments. Sibling objects using the same member name are unaffected — `{"x":{"p":1},"y":{"p":"s"}}` yields `X` and `Y`.

Ordering. [our choice] A module must be declared before it is referred to. A child's path is strictly longer than its parent's, so emitting deepest-first puts every child ahead of the parent that points at it.

An optional nested object makes both the key and the accessor optional, following the note's rule for nullable columns:

```
type t = { id : id; a : int; b : B.id option }
val b : t -> B.t option
```

6 Arrays

The note calls for repeated substructures to be normalized into separate relations but shows no example, so the shape below is ours. [our choice]

A parent cannot hold a single key to many children, so the direction reverses: the elements become a table whose rows carry a key back to the parent and an ordering index, and the parent holds *no column at all* — only an accessor returning a list.

```
{ "id_no": 1, "items": [ { "sku": "a", "qty": 2 },
                        { "sku": "b", "qty": 1 } ] }
```

```
(* generated.mli *)

module rec Items : sig
  type id
  type t = { id : id; parent_id : Root.id; idx : int; qty : int; sku : string }
  val get : id -> t
end

and Root : sig
  type id
  type t = { id : id; id_5fno : int }
  val get : id -> t
  val items : t -> Items.t list
end
```

Why module `rec`. An array makes the references cyclic: `Root` reaches `Items`, and `Items` carries a key back to `Root`. No ordering of plain module declarations satisfies both, so a schema containing any array is emitted as a mutually recursive group. A schema without arrays is emitted as plain modules, which is what keeps the note's own example byte-identical.

The index is not optional. JSON array order is significant. Without `idx`, `["a","b"]` and `["b","a"]` would shred identically and the order could not be recovered.

Arrays of scalars take the same shape, with a single `value` column in place of the element's members:

```
module rec Tags : sig
  type id
  type t = { id : id; parent_id : Root.id; idx : int; value : string }
```

```

val get : id -> t
end
and Root : sig ... val tags : t -> Tags.t list end

```

An array whose elements were never seen — empty in every document — gives the same table with no value column at all. A null element makes the value column optional: [1, null, 2] gives value : int option.

Absent and empty arrays are not distinguished. [our choice] There is no column in the parent for an array member, so there is nowhere to record that the member was missing rather than empty. Both give an empty list. This is a deliberate collapse, and the alternative — a presence column in the parent — would invent a column the note does not sanction.

7 What is refused

Everything below is our decision; the note does not discuss rejection at all. [our choice]

Input	Reported as
A top-level value that is not an object or an array of objects	<i>the top-level value is a number; expected an object, or an array of objects.</i>
A member holding two types with no common type	<i>the member .v holds both int and string, which have no common type.</i>
An array with an element that is an array	<i>the array at .m[] has an element that is itself an array. Wrap the inner array in an object to give it a name.</i>
An array mixing objects and scalars	<i>the array at .xs[] has both objects and scalars among its elements.</i>
A repeated member name	<i>the object at .b carries the member c more than once.</i>
A member colliding with a generated column (id, parent_id, idx)	<i>the member id collides with the generated id column.</i>
An object-valued member named get	<i>... it needs an accessor of that name, which collides with the generated get.</i>
Two tables that would take the same module name	<i>two tables would both be called Root.</i>

Nested arrays are refused for a structural reason rather than for convenience: every table is named for the member holding it, and an inner array sits in element position, which has no member name. There is nothing to name its table after. Wrapping the inner array in an object supplies the name and is accepted.

8 Names

[our choice] A member name is mangled into an OCaml identifier by a total function: a byte in a-z, A-Z, 0-9 or ' passes through; every other byte, *including* _, becomes _ followed by two lowercase hexadecimal digits; a result not beginning with a lowercase letter is prefixed f_; a result that is an OCaml keyword is suffixed _.

```

my-field -> my_2dfield Name -> f_Name
type -> type_1abc -> f_1abc

```

Escaping `_` is what makes the scheme injective, and injectivity is what stops two distinct members becoming one column. It also makes module names collision-free, since every mangled name begins with a lowercase letter and capitalising one character is then a bijection.

Injectivity is currently a property we rely on and have not proved. It is the first lemma worth doing.

9 The observation report

Running the binary with `--json` produces, alongside the module text, a per-table record of what was seen: how many times an object appeared at that path, and for each member how many times it held a value, an explicit `null`, and how often it was absent.

The denominator differs per table. The root table is visited once per document; a nested table once per parent that had it; an element table once per element. A ratio therefore means something different at each depth.

The report is where the `null`-versus-absent distinction survives, since the generated `option` cannot carry it.

10 Implementation

File	Holds
Tatami/Doc.lean	the document type and the JSON reader
Tatami/Path.lean	positions in the document; a path identifies a table
Tatami/Ty.lean	the types a member can have, and how two observations combine
Tatami/Infer.lean	reading JSON, walking objects and arrays, accumulating per member
Tatami/Schema.lean	the handoff: tables and their columns
Tatami/Mangle.lean	member names to OCaml identifiers
Tatami/Ocaml.lean	the output language
Tatami/Gen.lean	the note's correspondence, in code
Tatami/Print.lean	output language to text — the trusted step
Tatami/Error.lean	every rejection and its wording
Main.lean	the pipeline, the command line, the report

`Ty` and `Infer` know only JSON; `Ocaml` and `Print` know only OCaml; `Gen` is the only file that touches both, and `Schema` is what it passes between them.

The playground. `web/playground.html` is a page that shows the translation as it happens. Served by `dev/serve.py` it runs the Lean binary; opened without a server it falls back to an implementation of the same rules in JavaScript, and says which one produced what is on screen.

Running it both ways is a differential test. All twenty-five cases currently checked agree, including every rejection.

The output compiles. Every signature the generator has been run on has been checked with `ocamlc -c`, including the mutually recursive ones. This is a test, not a proof.

11 What can be guaranteed

Available now, about the generated code. That every record field name within a module is distinct and every referenced type is in scope — in short, that the emitted signature always

compiles. This is the guarantee worth having: a duplicate field name is the failure this generator is most likely to produce, and proving it requires the mangling lemma as a sub-goal.

Available now, about inference. That the schema does not depend on the order documents arrive in; that every value observed fits the column it was assigned; and that the assigned type is the least one that fits. The lattice facts these rest on (commutativity, idempotence, associativity of the join) are one-line proofs, since the type has few constructors.

Not available, and why. Nothing yet can be said about the data surviving, because **no data is produced**. The program emits a description of tables; it never emits a row. Three pieces are missing before a preservation statement can even be written: something that turns a document into rows, an interpretation of what `get` means, and something that walks rows back into a document.

The design note states in prose what that theorem should say: the original is no longer stored as a document, and is reconstructed by starting at a `Root.t` and calling accessors.

And it would need three exceptions. Even at present, three things are lost before a schema is formed: `null` versus a missing member; the order of members within an object; and how a number was written. A round-trip statement can only hold up to those.

12 Known gaps

1. **observeObject and the reader are partial.** Lean cannot see that a member of an object is smaller than the object, so the recursion is not structural, and the reader's recursion on a shrinking input is not visible either. A **partial** definition is opaque to proof, so `observeObject` must become total before anything about inference can be proved — a size measure on `Doc` and the lemma that members are smaller. This is now expressible, which it was not while the parser handed back a tree map. The reader is a trusted boundary either way, like the printer.
2. **Maps and recursive shapes are not handled.** An object whose keys are data rather than schema (`{"u_1001": { ... }}`) currently becomes a column per key. A self-referential shape becomes one table per observed depth. The note discusses neither; both need a decision before they need an implementation.
3. **Only one input form.** A single document, or a top-level array of them. JSON Lines and concatenated JSON are not read.